Section 9: Forests, Boosting, Splines and GAMs on Survey Data

GOVT 6039 | Fall 2026 | Discussion section, 11 November 2026

Table of contents

Getting this running 2

Part 1: the standard model **2**

Part 2: bagging and a random forest **4**

Part 3: boosting **6**

Part 4: splines, and what age does **7**

Part 5: a GAM, and what it cannot see **10**

Part 6: which model, and does it matter? **12**

Take-home **15**

Appendix: CatBoost **15**

References **17**

Lectures 16 and 17 estimate $\pi(x) = P(Y = 1 \mid X = x)$ for polling stations and use it to decide where to send 500 election monitors. Those data were simulated, which is why the lecture could print a row called "Truth". We have already worked with single trees. Today we apply everything that comes after them, **bagging and random forests, boosting, splines and GAMs**, to a real survey, where there is no "Truth" row. Nothing is computed by hand: each part fits a model, reads its output, and asks what it says about voters. The question: **how well can we tell who voted for Donald Trump in 2016 from who they are?**

Two reasons to care. A campaign has a voter file with background characteristics on everyone, and attitudes on almost no one. It has to decide which doors to knock on, so like the monitoring mission it needs an **order**: the people with the highest $\hat{\pi}$ should be the ones most likely to be its voters. And there is a live argument in political behaviour about the answer. Kim and Zilinsky (2024) show that demographics alone predict the presidential vote poorly: about 64% of votes correct across the ANES from 1952 to 2020, whether they use logit or random forests.

The data are the 2016 Cooperative Congressional Election Study (CCES), a national survey of 64,600 adults. We keep the 38,615 respondents who reported a presidential vote and answered every item we use. $Y_i = 1$ if respondent i says they voted for Trump, and 0 if they voted for anyone else. The analysis is unweighted, so everything below describes this sample of respondents. As in lecture, we start with six features:

Variable	Meaning
`age`	age in years
`female`	1 if a woman
`collegeed`	1 if the respondent has a college degree
`race`	White, Black, Hispanic, Asian, Other
`famincr`	family income, 1 (under $10,000) to 12 ($150,000 or more)
`lrelig`	religiosity scale built from four items (born-again, importance of religion, church attendance, prayer); mean 0, higher is more religious

The lecture trained on 5,000 stations. We train on a random 5,000 respondents and test on the other 33,615.

Getting this running

```
git clone --filter=blob:none --sparse
 ↪  https://github.com/ddekadt/cornell-govt-6039.git
cd cornell-govt-6039
git sparse-checkout set sections/section09
cd sections/section09
quarto render section09.qmd
```

The R packages are `ggplot2`, `patchwork`, `ranger`, `gbm`, `splines` and `mgcv`; the appendix also uses `catboost`, which is optional. Boosting here uses `gbm`, the package ISL uses, because its three arguments are exactly the lecture's three tuning parameters; the equivalent `xgboost` call is shown in Part 3. The data are in `data/cces16_vote.csv`, an extract of the 2016 CCES taken from the `TV16` data in Steven Miller's `stevedata` package.

Part 1: the standard model

```
cces <- read.csv("data/cces16_vote.csv")
cces$race <- factor(cces$race, levels = c("White", "Black", "Hispanic", "Asian",
 ↪  "Other"))

set.seed(6039)
```

```r
in_train <- sample(nrow(cces), 5000)
train <- cces[in_train, ]
test  <- cces[-in_train, ]
```

The baseline is the logit a political scientist would fit, each feature entered once:

```r
f <- trump ~ age + female + collegeed + race + famincr + lrelig

fit_logit <- glm(f, data = train, family = binomial)
p_logit   <- predict(fit_logit, test, type = "response")
round(summary(fit_logit)$coefficients[, 1:3], 3)
```

```
             Estimate Std. Error z value
(Intercept)    -0.307      0.153  -2.007
age             0.010      0.002   4.723
female         -0.595      0.067  -8.848
collegeed      -0.732      0.073 -10.007
raceBlack      -2.917      0.180 -16.169
raceHispanic   -0.660      0.129  -5.129
raceAsian      -0.853      0.230  -3.706
raceOther      -0.401      0.160  -2.511
famincr         0.038      0.012   3.305
lrelig          0.818      0.038  21.625
```

We score every model the same three ways on the test respondents: accuracy when we label $\hat{y}_i = 1\{\hat{\pi}_i \geq 0.5\}$; the AUC, the chance that a randomly chosen Trump voter gets a higher $\hat{\pi}$ than a randomly chosen other voter; and the share of Trump voters among the 1,000 respondents with the highest $\hat{\pi}$, which is what a campaign with 1,000 door-knocks cares about.

```r
acc   <- function(p, y = test$trump) mean((p >= 0.5) == y)
auc   <- function(p, y = test$trump) {
  n1 <- sum(y == 1); n0 <- sum(y == 0)
  (sum(rank(p)[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}
top_k <- function(p, k = 1000, y = test$trump) mean(y[order(-p)[1:k]])

round(c(share_trump = mean(test$trump), accuracy = acc(p_logit), auc =
 ↪  auc(p_logit), top_1000 = top_k(p_logit)), 3)
```

```
share_trump    accuracy         auc    top_1000
      0.409       0.704       0.770       0.819
```

1.1 The logit classifies 70% of test respondents correctly. Before deciding whether that is good: what would you score by labelling everybody "not Trump"? Is the result higher or lower than you expected from six characteristics that dominate election commentary?

1.2 The campaign takes the 1,000 highest $\hat{\pi}$. 82% of them are Trump voters, against 41% at random. Who are these people?

```
top <- test[order(-p_logit)[1:1000], ]
profile <- rbind(`Top 1,000`  = c(White = mean(top$race == "White"), College =
 ↪  mean(top$collegeed),
                                   Women = mean(top$female), Age = mean(top$age),
                                    ↪  Religiosity = mean(top$lrelig)),
                 `Everyone`   = c(mean(test$race == "White"),
                  ↪  mean(test$collegeed),
                                   mean(test$female), mean(test$age),
                                    ↪  mean(test$lrelig)))
round(profile, 2)
```

```
          White College Women   Age Religiosity
Top 1,000  0.98    0.04  0.14 64.80        1.44
Everyone   0.77    0.42  0.54 51.57       -0.01
```

Part 2: bagging and a random forest

2.1 Bagging grows each of 500 deep trees on a bootstrap sample of the training respondents, considers all $p = 6$ features at every split, and averages the 500 values of $\hat{\pi}$. A random forest does the same but considers only m randomly chosen features at each split. *Before the figure:* which will have the higher test AUC with 500 trees, and which term of the lecture's variance formula, $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, is your reason?

```
grow <- function(m) ranger(update(f, factor(trump) ~ .), data = train,
 ↪  probability = TRUE,
                            num.trees = 500, mtry = m, min.node.size = 10, seed =
                             ↪  6039,
                            importance = "permutation")
fit_bag <- grow(6)          # m = p: bagging
fit_rf  <- grow(2)          # m = 2: a random forest, as in lecture
p_bag <- predict(fit_bag, test)$predictions[, 2]
p_rf  <- predict(fit_rf,  test)$predictions[, 2]

auc_by_B <- function(fit, B = c(1, 2, 5, 10, 20, 50, 100, 200, 500)) {
  each <- predict(fit, test, predict.all = TRUE)$predictions[, 2, ]   # one
 ↪  column of pi-hat per tree
  data.frame(B = B, auc = sapply(B, function(b) auc(rowMeans(each[, 1:b, drop =
   ↪  FALSE]))))
}
```

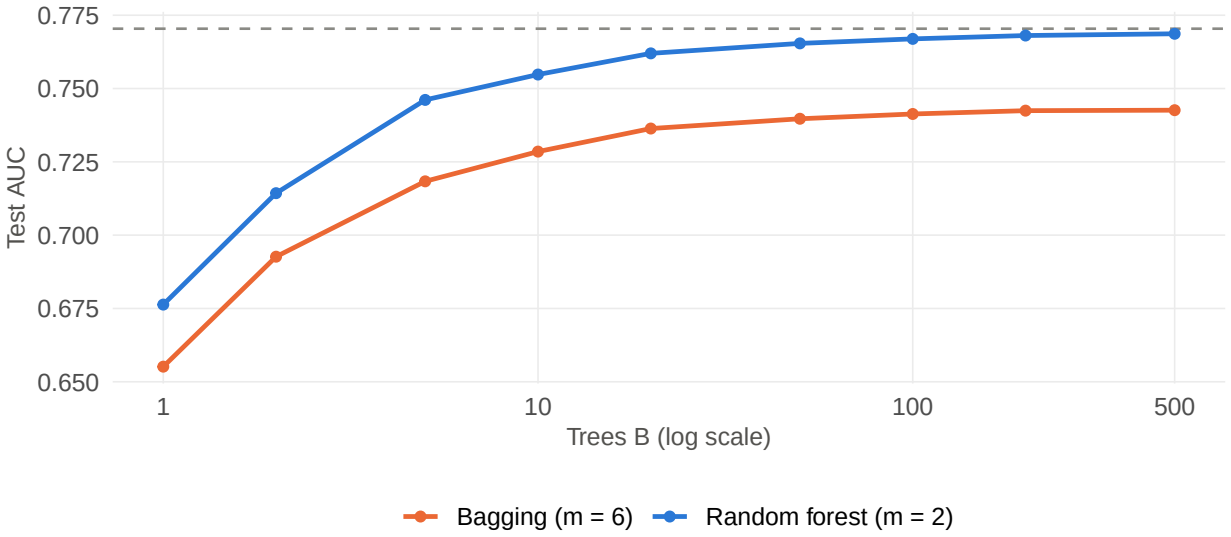


Figure 1: Test AUC as trees are added. Dashed line: the logit.

2.2 *(If time.)* We choose m on the out-of-bag respondents. Which m has the lowest out-of-bag Brier score?

```
oob <- data.frame(m = 1:6)
oob$oob_brier <- sapply(oob$m, function(m) grow(m)$prediction.error)
round(oob, 4)
```

```
  m oob_brier
1 1    0.2001
2 2    0.1896
3 3    0.1955
4 4    0.2012
5 5    0.2027
6 6    0.2034
```

2.3 A forest has no coefficients to read. The standard substitute is **permutation importance**: shuffle one feature among the out-of-bag respondents, so that it carries no information, and record how much worse the forest's predictions get. *Before the output:* rank the six features by how much you think the forest relies on them.

```
imp <- sort(fit_rf$variable.importance, decreasing = TRUE)
round(100 * imp / sum(imp), 1)                    # share of total importance,
 ↪  percent
```

```
   lrelig      race collegeed       age    female   famincr
     37.4      32.5      10.5       8.4       7.3       3.9
```

Part 3: boosting

A forest averages big trees grown independently. Boosting adds small trees one after another, each fitted to the residuals of the sum so far and added with a small step λ. The lecture's three tuning parameters are three `gbm` arguments: rounds $B =$ `n.trees`, step $\lambda =$ `shrinkage`, depth $d =$ `interaction.depth`.

```
boost <- function(lambda, d, B = 1500, form = f) {
  set.seed(6039)
  gbm(form, data = train, distribution = "bernoulli", n.trees = B, shrinkage =
   ↪  lambda,
      interaction.depth = d, n.minobsinnode = 10, cv.folds = 5, n.cores = 1,
       ↪  verbose = FALSE)
}
b_fast  <- boost(lambda = 0.10, d = 3)
b_slow  <- boost(lambda = 0.02, d = 3)
b_stump <- boost(lambda = 0.02, d = 1)
```

The same model with the lecture's `xgboost` (not run here):

```
X_tr <- model.matrix(f, train)[, -1]
fit_xgb <- xgboost(x = X_tr, y = factor(train$trump), nrounds = 500,
                   learning_rate = 0.02, max_depth = 3)
```

3.1 *Before the figure.* Sketch test AUC against rounds for $\lambda = 0.10$ and for $\lambda = 0.02$, both with depth 3. Which peaks first? What happens to each if we keep adding rounds? How is that different from adding trees to the forest in Part 2?

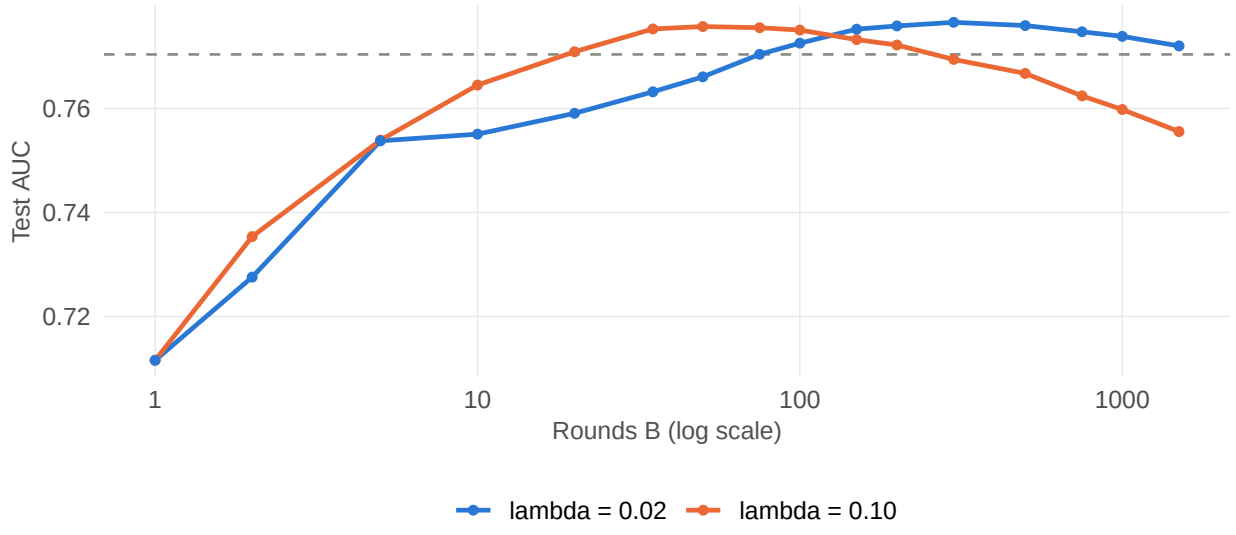


Figure 2: Test AUC as boosting rounds are added, depth 3. Dashed line: the logit. The test set is used here only to show the trade-off; B is chosen by CV on the training data.

```
B_cv <- function(fit) gbm.perf(fit, method = "cv", plot.it = FALSE)     # rounds
 ↪  with the lowest CV deviance
tune <- data.frame(model = c("lambda 0.10, depth 3", "lambda 0.02, depth 3",
 ↪  "lambda 0.02, depth 1"),
                   B_cv  = c(B_cv(b_fast), B_cv(b_slow), B_cv(b_stump)),
                   cv_deviance = c(min(b_fast$cv.error), min(b_slow$cv.error),
                    ↪  min(b_stump$cv.error)))
tune$test_auc_at_B_cv <- c(auc(predict(b_fast,  test, n.trees = tune$B_cv[1])),
                           auc(predict(b_slow,  test, n.trees = tune$B_cv[2])),
                           auc(predict(b_stump, test, n.trees = tune$B_cv[3])))
tune$test_auc_at_1500 <- c(auc(predict(b_fast,  test, n.trees = 1500)),
                           auc(predict(b_slow,  test, n.trees = 1500)),
                           auc(predict(b_stump, test, n.trees = 1500)))
knitr::kable(tune, digits = c(0, 0, 4, 4, 4))
```

model	B_cv	cv_deviance	test_auc_at_B_cv	test_auc_at_1500
lambda 0.10, depth 3	77	1.1061	0.7754	0.7556
lambda 0.02, depth 3	426	1.1051	0.7762	0.7720
lambda 0.02, depth 1	1052	1.1142	0.7725	0.7721

3.2 *(If time.)* Depth 1 trees split once, on one feature, so a sum of them cannot contain an interaction. Compare the depth 1 and depth 3 rows of the table. What does the difference tell you about these voters?

```
fit_boost <- b_slow
B_boost   <- tune$B_cv[2]
p_boost   <- predict(fit_boost, test, n.trees = B_boost, type = "response")

infl <- summary(fit_boost, n.trees = B_boost, plotit = FALSE)      # boosting's
 ↪  own importance measure
setNames(round(infl$rel.inf, 1), infl$var)
```

```
   lrelig      race       age collegeed    female   famincr
     39.3      31.4      12.7       7.4       5.9       3.2
```

Part 4: splines, and what age does

Importance told us that age matters, not how. A logit that is linear in age can only rise or fall at one pace. The points are the shares voting Trump in five-year age bins of the training data.

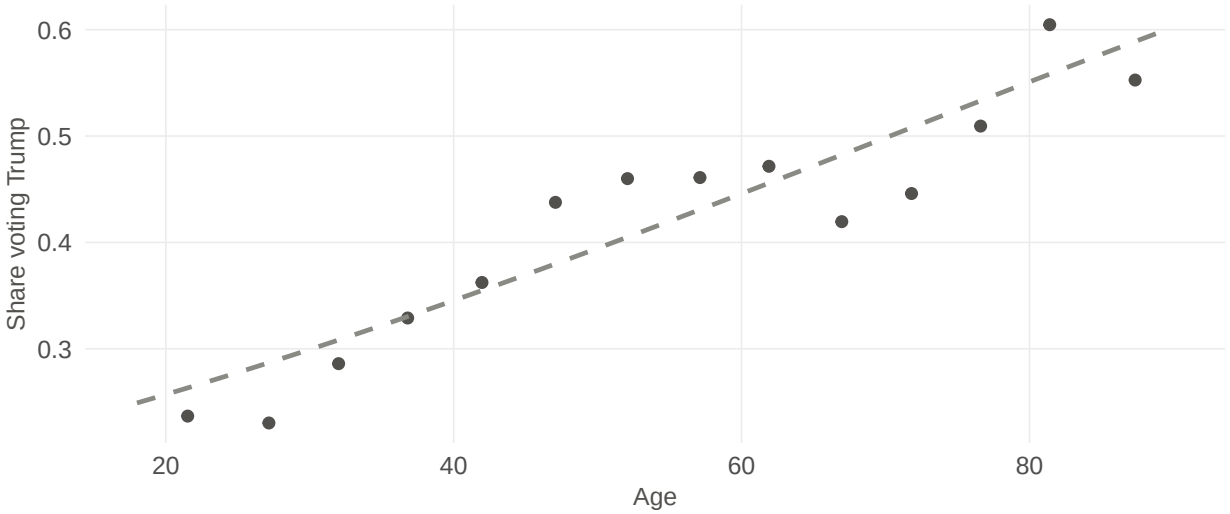


Figure 3: Share voting Trump by age in the training data (five-year bins), with a logit that is linear in age.

A spline lets the logit bend: `ns(age, df = 4)` replaces the single `age` column with four columns, and `glm()` fits a logit on them as on any others. The one decision is how flexible to make it. As in lecture, we choose the degrees of freedom of a natural spline by 10-fold CV on the training data.

4.1 *Before the figure.* df $= 1$ is the straight line above. What do you expect the CV log loss to do as df goes from 1 to 10? Then read the right panel as an analyst: describe how support for Trump changes with age.

```
log_loss <- function(p, y) -mean(y * log(p) + (1 - y) * log(1 - p))

set.seed(6039)
fold <- sample(rep(1:10, length.out = nrow(train)))
cv_loss <- sapply(1:10, function(df) mean(sapply(1:10, function(k) {
  fit <- glm(trump ~ ns(age, df = df), data = train[fold != k, ], family =
 ↪  binomial)
  log_loss(predict(fit, train[fold == k, ], type = "response"), train$trump[fold
   ↪  == k])
})))
df_best <- which.min(cv_loss)
df_best
```

```
[1] 4
```

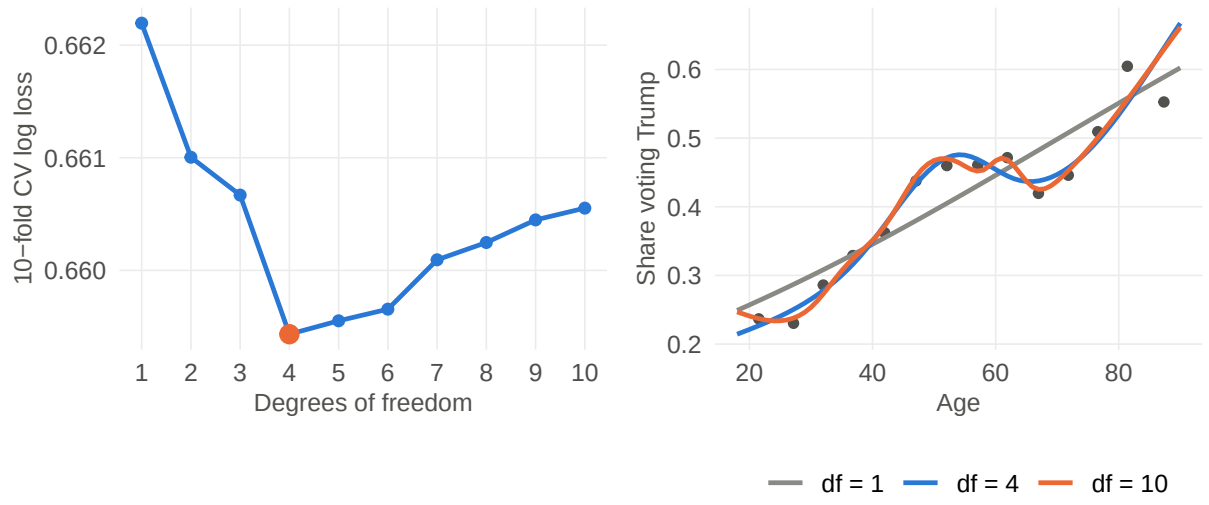


Figure 4: Left: 10-fold CV log loss of a natural spline in age, by degrees of freedom. Right: the fitted curves for df = 1, the CV choice, and df = 10.

4.2 *(If time.)* The other route in lecture: use a rich basis and let a penalty on curvature decide how much the curve may bend. In `gam()`, `sp` fixes λ and `method = "REML"` estimates it. Which of the three curves will wiggle? What will the effective degrees of freedom be under the huge penalty?

```
s_small <- gam(trump ~ s(age, k = 30), data = train, family = binomial, sp =
 ↪  1e-6)
s_large <- gam(trump ~ s(age, k = 30), data = train, family = binomial, sp = 1e6)
s_reml  <- gam(trump ~ s(age, k = 30), data = train, family = binomial, method =
 ↪  "REML")
edf1 <- function(fit) sum(fit$edf) - 1                # edf of the curve,
 ↪  without the intercept
round(c(small_penalty = edf1(s_small), large_penalty = edf1(s_large), REML =
 ↪  edf1(s_reml)), 1)
```

```
small_penalty large_penalty          REML
         28.9           1.0           4.9
```

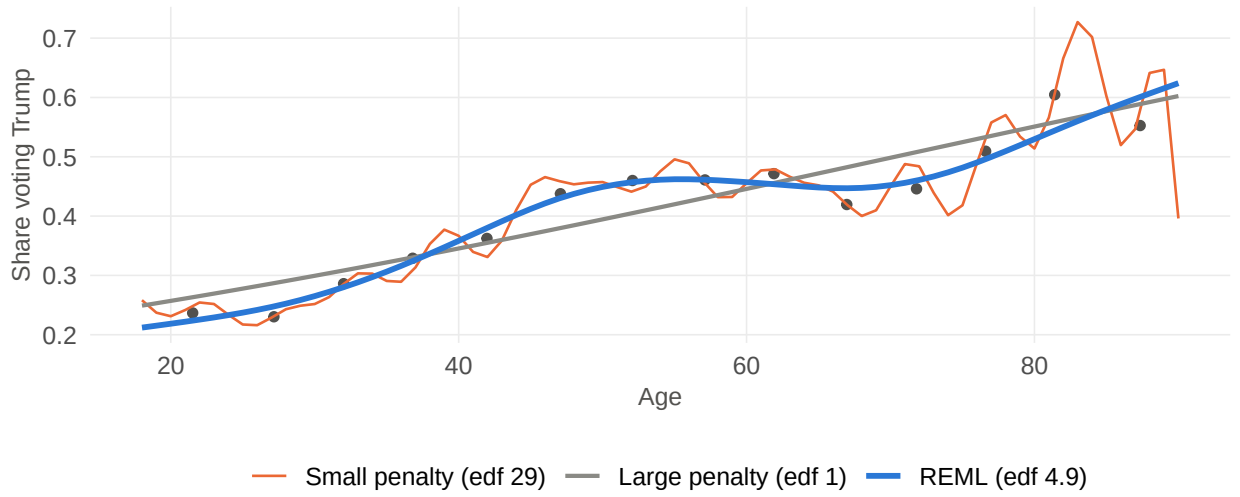


Figure 5: A smoothing spline in age under a tiny penalty, a huge penalty, and the penalty chosen by REML.

Part 5: a GAM, and what it cannot see

Now all six features in one interpretable model. Each continuous feature gets its own smoothing spline, the curves are added up on the log-odds scale, and REML chooses every penalty at once. The rest enter as in the logit.

```
fit_gam <- gam(trump ~ female + collegeed + race + s(age) + s(famincr, k = 8) +
 ↪  s(lrelig),
               data = train, family = binomial, method = "REML")
p_gam <- predict(fit_gam, test, type = "response")
round(summary(fit_gam)$s.table[, c("edf", "Chi.sq", "p-value")], 3)
```

```
             edf  Chi.sq p-value
s(age)     4.520  50.400   0.000
s(famincr) 2.084  11.137   0.009
s(lrelig)  2.097 461.082   0.000
```

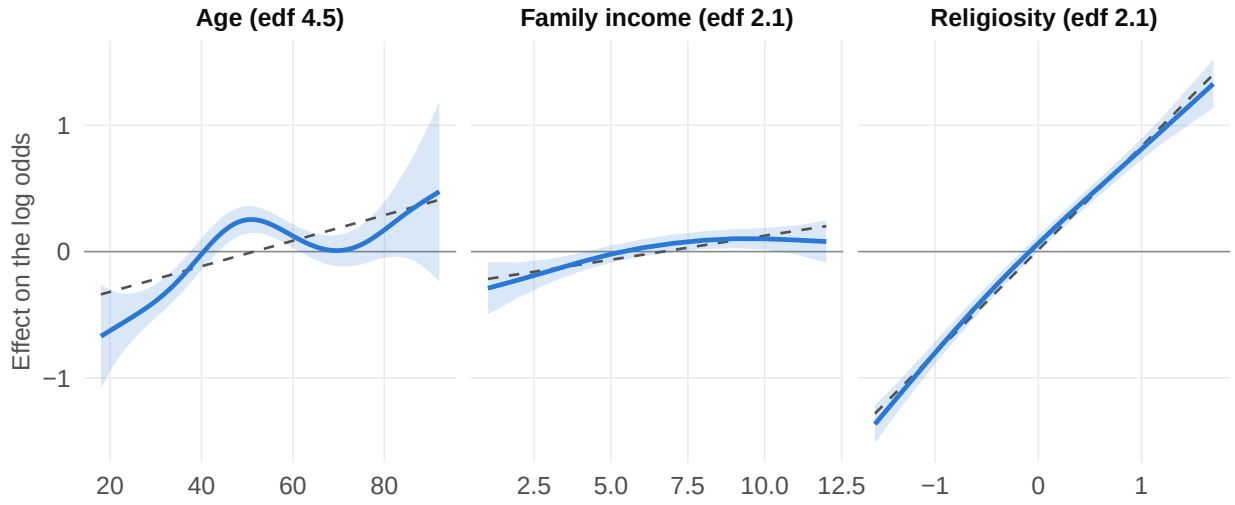


Figure 6: The three smooths with 95% bands, on the log-odds scale, holding the other features fixed. Dashed: the straight line the logit fits, centred the same way.

5.1 Read the edf column and the figure together. For which features did the data ask for a curve? Is the age curve the one from Part 4, now that the other five features are held fixed? What does the logit's single coefficient on `age` (0.010 per year) get wrong?

5.2 Now a feature the GAM treats exactly as the logit does. The "diploma divide" is the claim that a college degree moved voters away from Trump. Both the logit and the GAM give `collegeed` one coefficient (-0.73 in the logit). *Before the table:* does one coefficient mean the same change in $\hat{\pi}$ for White, Black and Hispanic respondents? For each model, we take every test respondent of a given race, set `collegeed` to 0 and then to 1, and average the difference in $\hat{\pi}$. The first column is the raw gap in the test data.

```
college_effect <- function(pred, r) {
  lo <- hi <- test[test$race == r, ]; lo$collegeed <- 0; hi$collegeed <- 1
  mean(pred(hi) - pred(lo))
}
models <- list(Logit    = function(d) predict(fit_logit, d, type = "response"),
               GAM      = function(d) predict(fit_gam, d, type = "response"),
               Forest   = function(d) predict(fit_rf, d)$predictions[, 2],
               Boosting = function(d) predict(fit_boost, d, n.trees = B_boost,
                ↪  type = "response"))
races <- c("White", "Black", "Hispanic")
raw   <- sapply(races, function(r) mean(test$trump[test$race == r &
 ↪  test$collegeed == 1]) -
                                    mean(test$trump[test$race == r &
                                     ↪  test$collegeed == 0]))
eff   <- sapply(models, function(m) sapply(races, function(r) college_effect(m,
 ↪  r)))
round(100 * cbind(`Raw gap` = raw, eff), 1)          # percentage points
```

```
          Raw gap Logit   GAM Forest Boosting
White       -19.2 -15.4 -15.0  -15.2    -14.1
Black        -0.4  -4.4  -4.3    0.2     -0.5
Hispanic     -0.5 -14.0 -13.5    3.3     -1.6
```

The repair is to say so in the model. A GAM is still a model we write down:

```
fit_gam_x <- gam(trump ~ female + collegeed * race + s(age) + s(famincr, k = 8) +
 ↪  s(lrelig),
                 data = train, family = binomial, method = "REML")
p_gam_x <- predict(fit_gam_x, test, type = "response")

round(summary(fit_gam_x)$p.table[grep("collegeed",
 ↪  rownames(summary(fit_gam_x)$p.table)), 1:2], 2)
```

```
                       Estimate Std. Error
collegeed                 -0.87       0.08
collegeed:raceBlack        0.37       0.40
collegeed:raceHispanic     1.26       0.26
collegeed:raceAsian        1.62       0.61
collegeed:raceOther        0.58       0.32
```

```
eff_x <- sapply(races, function(r) college_effect(function(d) predict(fit_gam_x,
 ↪  d, type = "response"), r))
round(100 * eff_x, 1)
```

```
   White    Black Hispanic
   -18.2     -3.1      7.6
```

Part 6: which model, and does it matter?

```
fit_ns <- glm(trump ~ female + collegeed + race + ns(age, df = df_best) +
 ↪  ns(famincr, df = 3) + ns(lrelig, df = 3),
              data = train, family = binomial)
p_ns   <- predict(fit_ns, test, type = "response")

preds <- list(Logit = p_logit, Bagging = p_bag, `Random forest` = p_rf, Boosting
 ↪  = p_boost,
              `Spline logit` = p_ns, GAM = p_gam, `GAM + college x race` =
               ↪  p_gam_x)

set.seed(6039)                                          # bootstrap the test
 ↪  respondents
```

```r
boot <- replicate(300, {
  i <- sample(nrow(test), replace = TRUE)
  sapply(preds, function(p) auc(p[i], test$trump[i]))
})
ci  <- apply(boot, 1, quantile, c(0.025, 0.975))
dif <- apply(boot - boot[rep("Logit", nrow(boot)), ], 1, quantile, c(0.025,
 ↪  0.975))

score <- data.frame(Model = names(preds),
                    `Test AUC` = f3(sapply(preds, auc)),
                    `95% interval` = sprintf("[%.3f, %.3f]", ci[1, ], ci[2, ]),
                    `Minus logit` = sprintf("[%+.3f, %+.3f]", dif[1, ], dif[2,
                     ↪  ]),
                    Accuracy = f3(sapply(preds, acc)),
                    `Top 1,000` = f2(sapply(preds, top_k)), check.names = FALSE)
score$`Minus logit`[1] <- ""
knitr::kable(score, row.names = FALSE, align = "lrrrrr")
```

Model	Test AUC	95% interval	Minus logit	Accuracy	Top 1,000
Logit	0.770	[0.766, 0.775]		0.704	0.82
Bagging	0.743	[0.737, 0.747]	[-0.031, -0.024]	0.687	0.78
Random forest	0.769	[0.764, 0.773]	[-0.004, +0.000]	0.706	0.81
Boosting	0.776	[0.772, 0.781]	[+0.004, +0.007]	0.708	0.82
Spline logit	0.772	[0.768, 0.777]	[+0.001, +0.003]	0.707	0.81
GAM	0.773	[0.769, 0.777]	[+0.001, +0.003]	0.706	0.81
GAM + college x race	0.773	[0.769, 0.778]	[+0.001, +0.004]	0.707	0.81

"Minus logit" is the 95% interval for the difference in AUC from the logit. Accuracy is at a threshold of 0.5. "Top 1,000" is the share of Trump voters among the 1,000 test respondents with the highest $\hat{\pi}$.

6.1 In lecture the forest and boosting beat the logit clearly (0.858 against 0.825). What is the ordering here, and how large are the gaps? With 33,615 test respondents the intervals are narrow: which differences from the logit are distinguishable from zero? Which would change what the campaign does?

6.2 We trained on 5,000 respondents. A typical poll has 1,000; a campaign's modelling file may have 20,000 or more. The figure refits each model on training sets of different sizes and scores it on the same 10,000 held-out respondents (five random training sets per size; the code is in `learning_curve.R`). Which model would you use with a poll of 500? Which with 20,000 respondents? Why does the answer change?

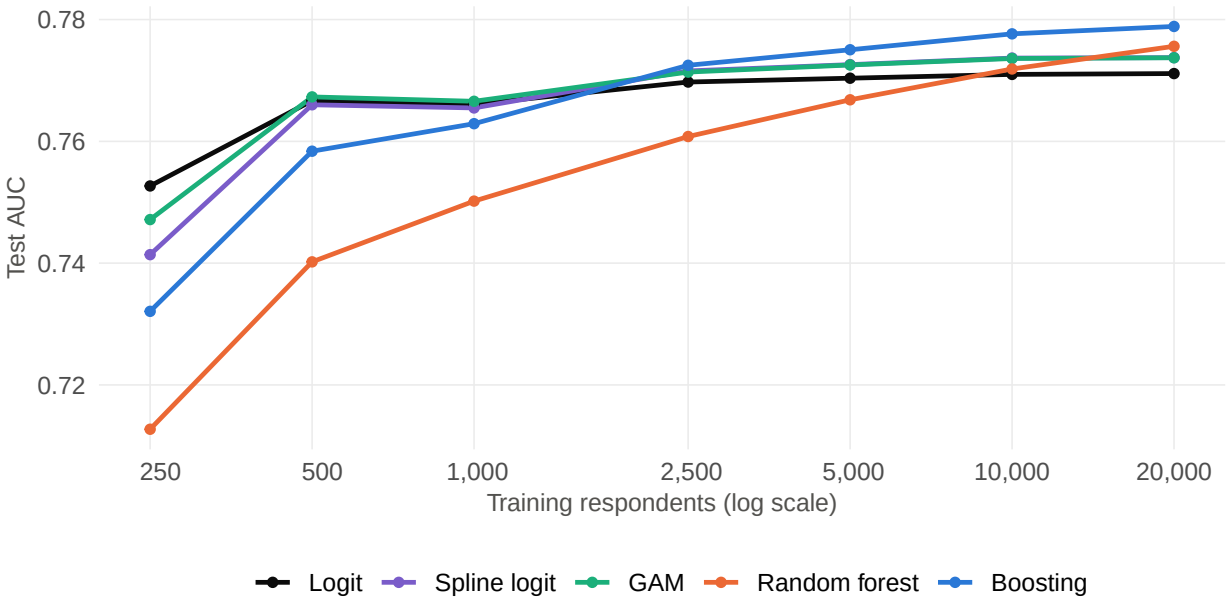


Figure 7: Test AUC against training-set size, averaged over five random training sets of each size.

6.3 Everything so far used the same six background features. The survey also asked about politics. We add four items: seven-point party identification (`pid7`, 1 strong Democrat to 7 strong Republican), five-point ideology (`ideo`, 1 very liberal to 5 very conservative), and two racial-attitude scales built from four agree/disagree items (`lcograc`: racism is rare, whites have advantages; `lemprac`: fear of other races, anger that racism exists). *Before the table:* every model so far gave an AUC of about 0.77. Write down your guess for a plain logit with these four items added, and for a random forest on the same ten features. Then: which mattered more today, the choice of model or the choice of features? Why can the campaign not simply use the bottom rows?

```
f_att <- update(f, . ~ . + pid7 + ideo + lcograc + lemprac)

fit_logit_att <- glm(f_att, data = train, family = binomial)
fit_rf_att    <- ranger(update(f_att, factor(trump) ~ .), data = train,
 ↪  probability = TRUE,
                        num.trees = 500, min.node.size = 10, seed = 6039)
fit_pid       <- glm(trump ~ pid7, data = train, family = binomial)

p_logit_att <- predict(fit_logit_att, test, type = "response")
p_rf_att    <- predict(fit_rf_att, test)$predictions[, 2]
p_pid       <- predict(fit_pid, test, type = "response")
```

Model	Test AUC	Accuracy	Top 1,000
Six background features, logit	0.770	0.704	0.82
Six background features, boosting	0.776	0.708	0.82
Party identification alone, logit	0.911	0.858	0.94

Model	Test AUC	Accuracy	Top 1,000
Background + four attitude items, logit	0.948	0.886	0.98
Background + four attitude items, forest	0.951	0.888	0.98

Take-home

1. Tune the boosted model properly: a grid over `shrinkage`, `interaction.depth` and `n.trees` with 5-fold CV. How much test AUC do you gain over the settings used here?
2. Refit the boosted model with `xgboost`. Do you land in the same place as `gbm`?
3. Part 5 found one interaction by comparing the GAM with the forest and boosting. Look for another, for example religiosity by race: add `s(lrelig, by = race)` to the GAM and check whether REML keeps separate curves.
4. Repeat 4.1 for `lrelig` and `famincr`. Does CV choose more than one degree of freedom for either?

Appendix: CatBoost

This appendix is not part of the 50 minutes. CatBoost (Prokhorenkova et al. 2018) is a gradient boosting library, like `gbm` and `xgboost`: small trees added one after another, each fitted to what the current model gets wrong. It differs in three ways, and the first is the one that matters most for survey data.

1. **Categorical features without dummies.** A survey is full of categories with many levels: state, occupation, denomination, party voted for, interviewer. The usual options are both poor. Dummies give a 51-level feature 50 columns, most of them nearly empty. Replacing each category by its mean outcome ("target encoding") uses each respondent's own y_i to build that respondent's own feature, which leaks the outcome into x. CatBoost puts the training rows in a random order and encodes each row's category using only the rows *before* it: the share of $y = 1$ among earlier respondents in the same category, pulled toward the overall share when there are few of them. This is called an ordered target statistic. One number per category, no respondent's outcome in their own feature, and small categories shrunk toward the mean.
2. **Ordered boosting.** The same idea applied to the residuals. Ordinary boosting computes each row's residual from a model that was trained on that row, so the residuals in training are systematically smaller than they would be on new data. CatBoost can compute row i's residual from a model fitted only to rows before i (`boosting_type = "Ordered"`).
3. **Symmetric trees.** Every node at the same depth of a tree uses the same split, so a depth-6 tree asks six questions and has 64 leaves. This is a strong constraint on each tree, which makes the model harder to overfit and fast to evaluate.

In practice the selling points are that categorical features work without preprocessing and that the defaults (1,000 trees of depth 6, with a learning rate chosen automatically) are usually close to what tuning would give. It is most useful when several important features are categories with many levels and the data are not huge. When every feature is numeric it is one more boosting library.

The R package is not on CRAN. Download the build for your system from `github.com/catboost/catboost/releas` (the Linux file is `catboost-R-linux-x86_64-1.2.10.tgz`) and install it with `R CMD INSTALL`. The chunks below run only if `catboost` is installed.

The exercise. Where someone lives should tell us something about their vote, and the survey records each respondent's `state`: 51 categories. With 5,000 training respondents, the median state has 65 of them and the smallest has 4.

A.1 We add `state` to the logit (as 50 dummies) and to the boosted model from Part 3 (as a factor). *Before the table:* will the test AUC go up?

```
train$state <- factor(train$state)
test$state  <- factor(test$state, levels = levels(train$state))
f_state <- update(f, . ~ . + state)

p_logit_state <- predict(glm(f_state, data = train, family = binomial), test,
 ↪  type = "response")
b_state       <- boost(lambda = 0.02, d = 3, form = f_state)
p_boost_state <- predict(b_state, test, n.trees = B_cv(b_state), type =
 ↪  "response")
```

Table 5: Test AUC without and with state.

Model	Six features	Six features + state
Logit	0.7704	0.7680
Boosting (gbm)	0.7762	0.7708

A.2 Now CatBoost, three ways: the six features; the six features plus `state`, declared categorical simply by being a factor; and the same with CatBoost's target statistics switched off, so that `state` is expanded into dummies (`one_hot_max_size = 255`). What does the comparison between the second and third rows tell you?

```
library(catboost)

cb_fit <- function(vars, ...) {
  pool <- catboost.load_pool(train[, vars], label = train$trump)       # factor
 ↪  columns are treated as categorical
  fit  <- catboost.train(pool, params = list(loss_function = "Logloss",
 ↪  random_seed = 6039, thread_count = 1,
                                             logging_level = "Silent", ...))
  list(fit = fit, pool = pool,
       p = catboost.predict(fit, catboost.load_pool(test[, vars]),
        ↪  prediction_type = "Probability"))
}
six <- all.vars(f)[-1]
cb_six    <- cb_fit(six)                                          # defaults, six
 ↪  features
```

```
cb_state  <- cb_fit(c(six, "state"))                              # state through
 ↪  ordered target statistics
cb_onehot <- cb_fit(c(six, "state"), one_hot_max_size = 255)     # state as
 ↪  dummies

imp_cb <- catboost.get_feature_importance(cb_state$fit, cb_state$pool)[, 1]
names(imp_cb) <- c(six, "state")
```

Model	Test AUC
CatBoost, six features	0.7705
CatBoost, six features + state (target statistics)	0.7751
CatBoost, six features + state (dummies)	0.7693

```
   lrelig      race     state       age   famincr    female collegeed
     24.7      17.8      17.7      15.7       9.4       7.5       7.2
```

A.3 *(If time.)* CatBoost is often described as needing no tuning. Compare its default fit on the six features with the models from the section: logit 0.7704, `gbm` with CV-chosen rounds 0.7762, GAM 0.7728. What do you conclude about when to reach for it?

References

- Ansolabehere, S. & Schaffner, B. F. (2017) *Cooperative Congressional Election Study, 2016: Common Content*. Harvard Dataverse.
- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed., 7.3–7.7 and 8.2.
- Kim, S. S. & Zilinsky, J. (2024) "Division Does Not Imply Predictability: Demographics Continue to Reveal Little About Voting and Partisanship." *Political Behavior* 46(1): 67–87.
- Miller, S. V. `stevedata`: Steve's Toy Data for Teaching About a Variety of Methodological, Social, and Political Topics. R package (the `TV16` data). github.com/svmiller/stevedata.
- Montgomery, J. M. & Olivella, S. (2018) "Tree-Based Models for Political Science Data." *American Journal of Political Science* 62(3): 729–744.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. & Gulin, A. (2018) "CatBoost: Unbiased Boosting with Categorical Features." *Advances in Neural Information Processing Systems* 31.